

Feature Toggle Dynamics in Large-Scale Systems: Prevalence, Growth, Lifespan, and Benchmarking

Xhevahire Tërnav

LTCI, Télécom Paris, Institut Polytechnique de Paris

xhevahire.ternava@telecom-paris.fr

VARIABILITY 2026, Limassol, Cyprus

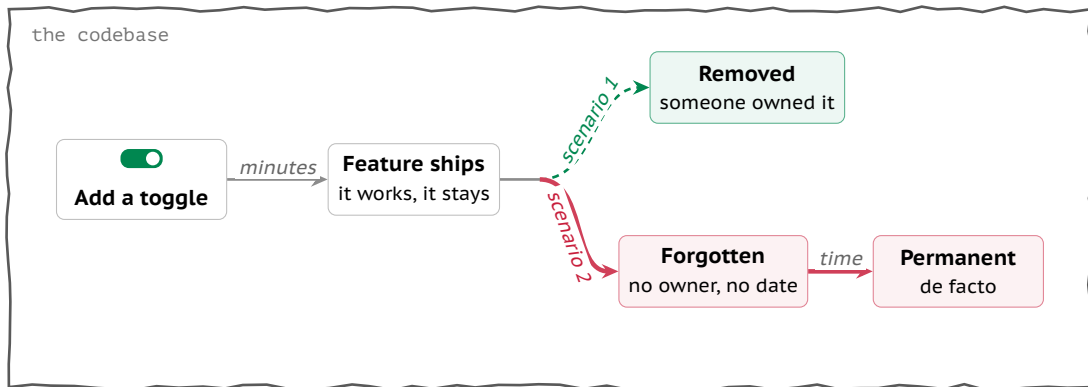
October 2, 2026

Easy to add, easy to forget

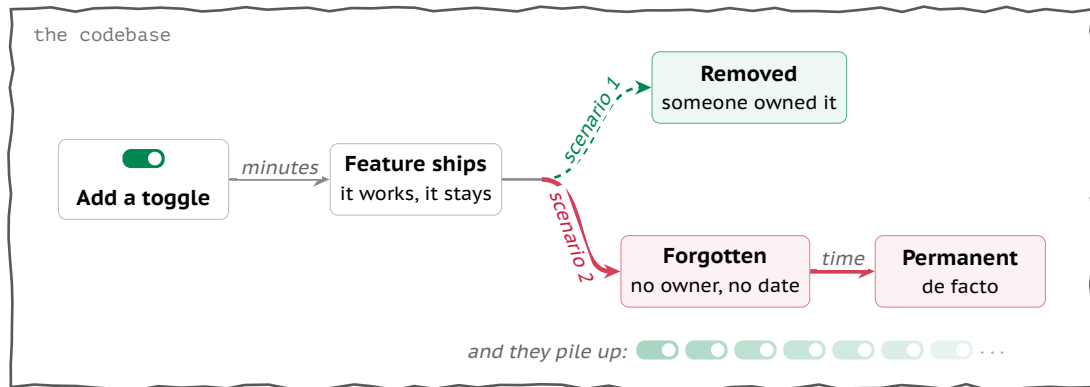
the codebase



Easy to add, easy to forget



Easy to add, easy to forget



Toggles become permanent **not by design but by neglect**, accumulating as invisible **technical debt**.

Inside that codebase, one real feature gate

the codebase

kube_features.go

```
// owner @bswartz  
// GA in 1.33, so remove the gate in 1.36  
AnyVolumeDataSource: {Version: "1.33", PreRelease: GA}
```

← The removal deadline lives in a comment. Nothing in the build enforces it.

pkg/volume/pvc_protection.go

```
if gate.Enabled(AnyVolumeDataSource) {  
    ...  
}
```

← Every call site adds a branch, so the number of paths to test keeps growing.

and the same gate is read in

pkg/apis

pkg/registry

test/e2e

...

Cheap to declare. Costly to retire. That asymmetry is what makes toggles accumulate.

Related work and the research gap

Prior work covers toggle management [1,2,3], interactions [4,5], complexity [6], usage patterns [7], addition and removal practices [8,9], underlying rationales [10,11], standardization [12], and cleanup tooling, such as Piranha [13].

Still unanswered:

- How *many* toggles live in modern codebases?
- Are they added faster than they are removed?
- Do active toggles *accumulate* over time?
- How *long* do individual toggles survive?

The gap: no **longitudinal** study has quantified how toggles evolve over time across organizational contexts, so teams have *no benchmark* that tells healthy management of feature toggles apart from their silent accumulation.

Three research questions

RQ1 Additions versus removals. Does cleanup keep pace with additions?

RQ2 Active inventory. Do active toggles grow, stabilise, or decline?

RQ3 Lifespan. How long do toggles survive, and how does that differ across projects?

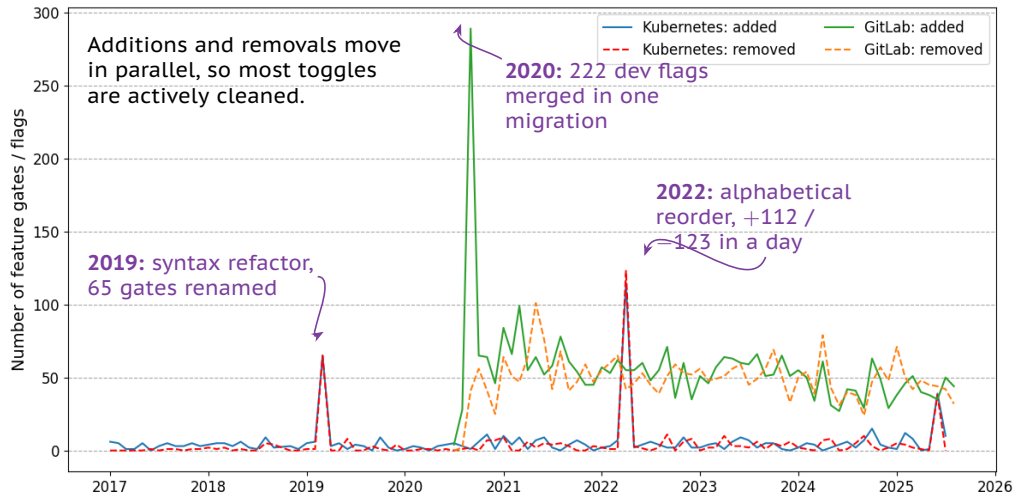
Approach: we mine the **version control history**, track every toggle addition and removal, then compute how the active inventory accumulates and how long toggles survive before removal (**Kaplan-Meier survival curves**).

Two large-scale systems, two very different rhythms

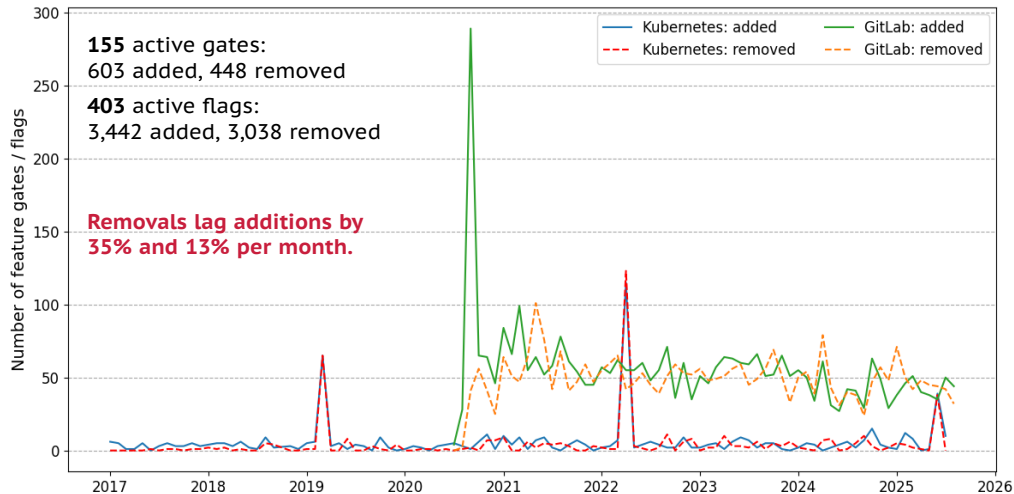
	Kubernetes	GitLab
Domain	Container orchestration	DevOps platform
Language	Go	Ruby on Rails
Code size	about 10 MLoC	about 5 MLoC
History mined	8.5 years of feature gates	5 years of feature flags
Release cadence	Quarterly	Monthly
Active toggles	155 gates	403 flags

Mined from [kube_features.go](#) and [config/feature_flags/](#), then validated against the project documentation and the command line output.

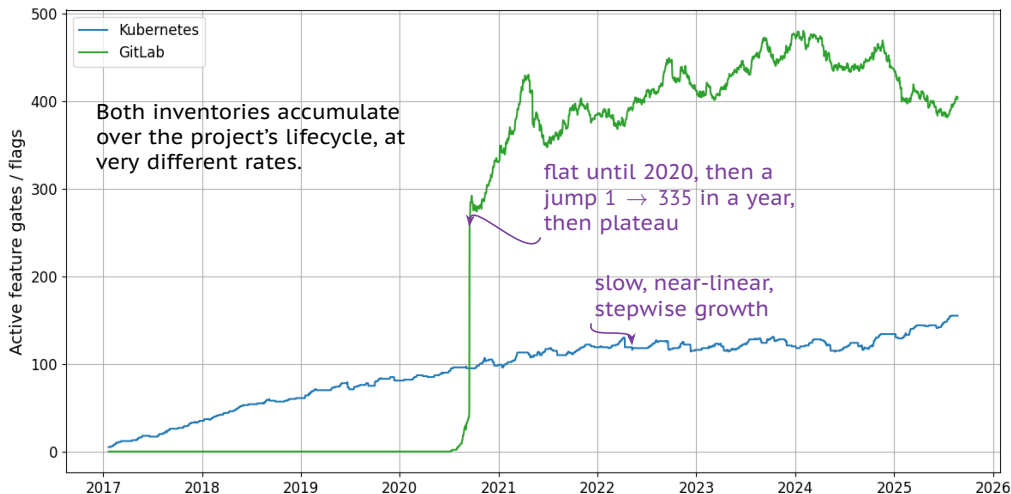
RQ1. Do removals keep pace with additions?



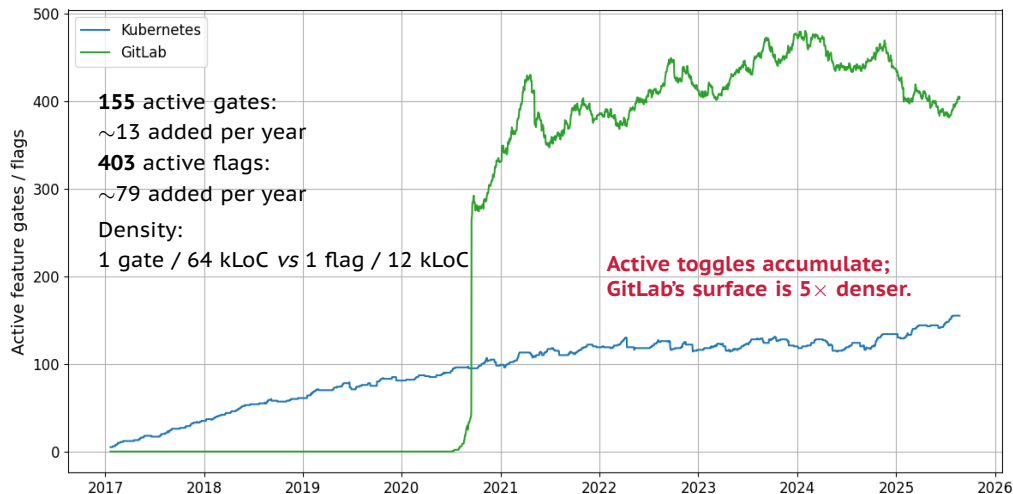
RQ1. Do removals keep pace with additions?



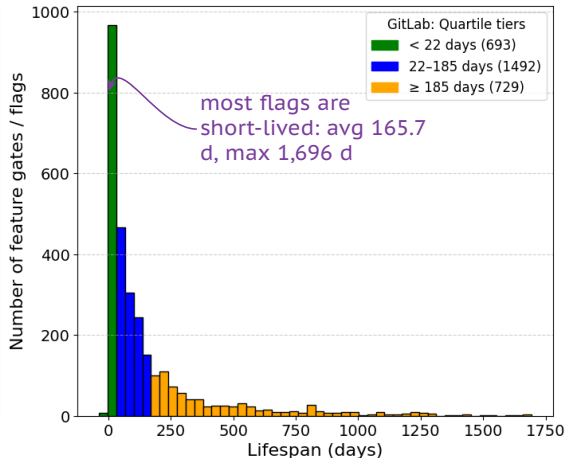
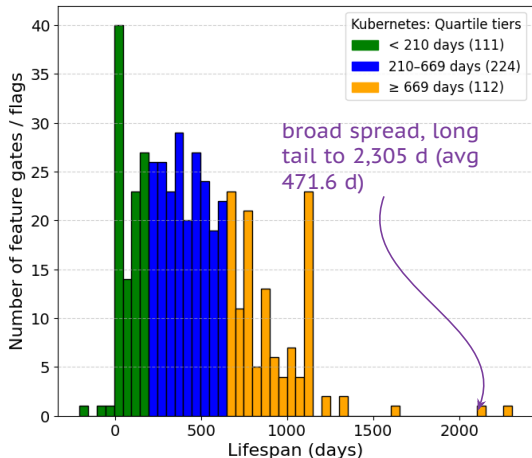
RQ2. Do active toggles accumulate?



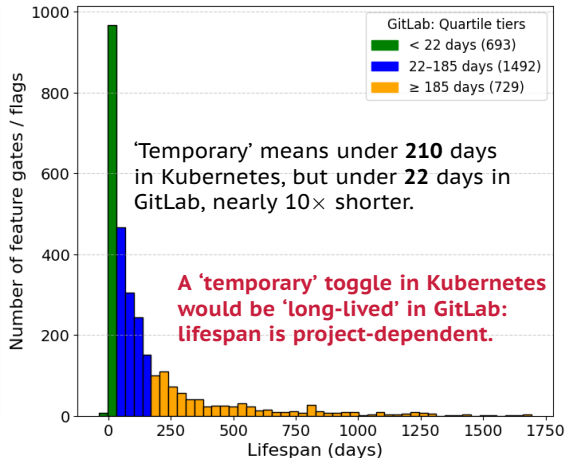
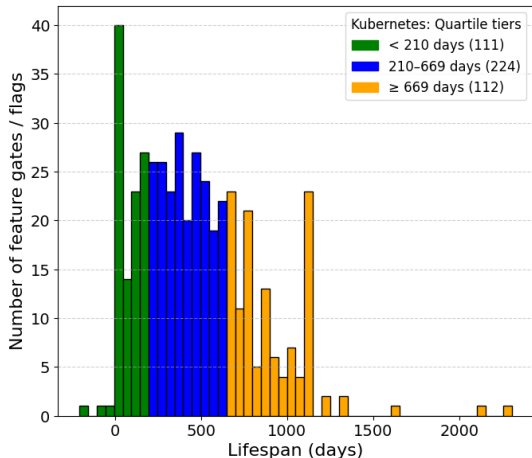
RQ2. Do active toggles accumulate?



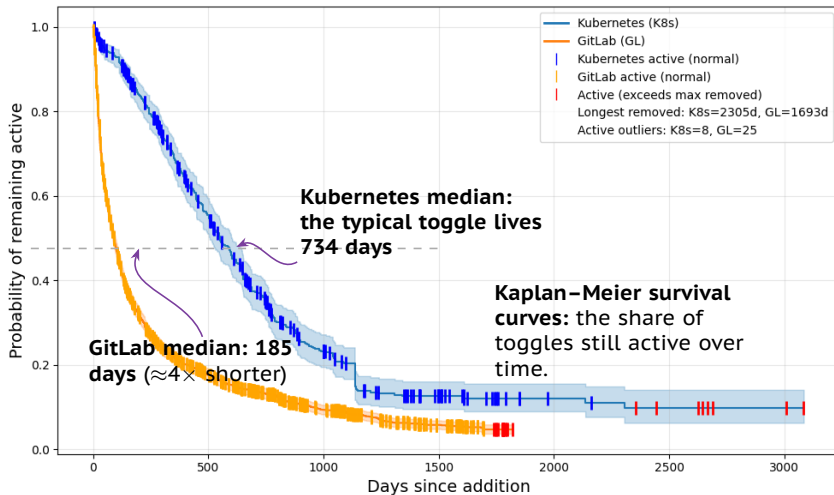
RQ3. Lifespan: “Temporary” means very different things



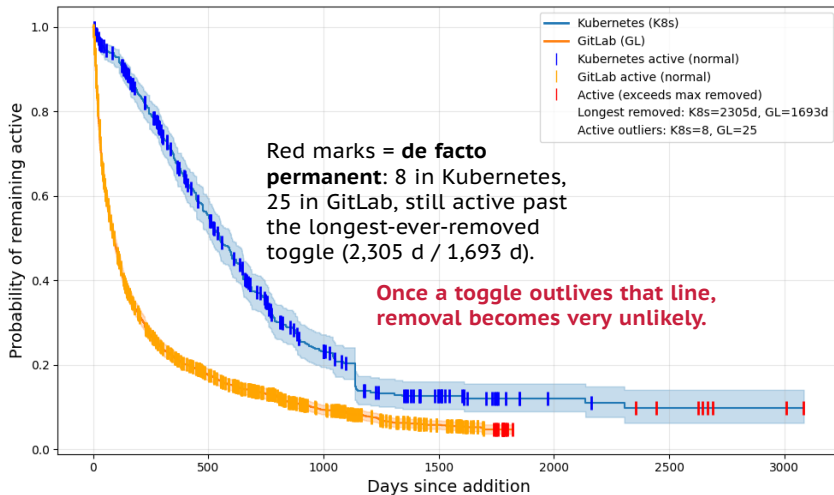
RQ3. Lifespan: “Temporary” means very different things



RQ3. Lifespan: Survival and the de facto permanent toggles



RQ3. Lifespan: Survival and the de facto permanent toggles



From findings to a benchmark: five metrics

How can practitioners determine whether their feature toggle inventory is healthy or problematic?

- **Churn rate** C , additions plus removals per month
- **Net accumulation** N , additions minus removals per month
- **Cleanup ratio** R , removed over added
- **toggle Density** D , active toggles per kLoC
- **normalized lifeSpan** S , median over release cycle

Empirically derived threshold zones

Each metric maps to **low**, **moderate**, and **high** zones, calibrated from the two systems, with the Kubernetes and GitLab baselines underneath.

A self calibrating insight: in *days* the two lifespans differ by a factor of four, but in **release cycles** both converge to about **six**. Release cadence explains most of the difference.

benchmark comparison

threshold reference

saved projects

Thresholds derived from Kubernetes and GitLab.

metric	low	moderate	high
C	< 15 Deliberate	15-100 Balanced	≥ 100 Rapid
N	< 2 Sustainable	2-5 Warning	≥ 5 Critical
R	≥ 0.85 Healthy	0.70-0.85 Warning	< 0.70 Debt
D	< 0.02 Low	0.02-0.10 Typical	≥ 0.10 High
S	< 3 Short	3-8 Typical	≥ 8 Long

Baseline values (up to August 2025).

metric	◆ Kubernetes	■ GitLab
churn rate	10.2/month	104.5/month
net accumulation	1.5/month	6.5/month
cleanup ratio	0.74	0.88
toggle density	0.016/kLoC	0.081/kLoC
norm. lifespan	6.1 cycles	6.2 cycles

Two valid toggle management strategies, neither one is wrong

Low volume and long tail (*the Kubernetes style*)

few toggles, added sparingly, kept across many releases,
long and staged deprecation

High volume and short lived (*the GitLab style*)

many disposable switches, rapid iteration and A/B tests,
needs aggressive cleanup

- Our findings do not indicate that either approach is inherently superior.
- We believe that each reflects organizational practices and development rhythms tailored to the project's domain, release cycle, and acceptable failure cost.

Try it: a live benchmarking dashboard

- **Compare:** a radar of Kubernetes against GitLab across the five metrics.
- **Self assess:** enter your project's numbers and see which zones you land in.
- **Contribute:** submit your data to grow the shared benchmark.

● ternava.github.io/detog/

Your project metrics.

project name

active feature toggles

total toggles added

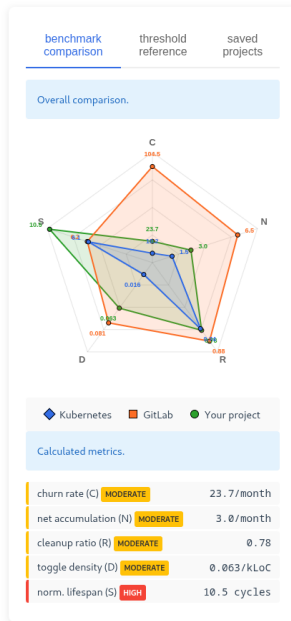
total toggles removed

lines of code (LoC)

analysis period (months)

median toggle lifespan (days)

average release cycle (days)



Summary: Feature Toggle Dynamics in Large-Scale Systems

Prevalence, Growth, Lifespan, and Benchmarking

1. Toggles are **prevalent** (155 active gates in Kubernetes and 403 active flags in GitLab).
2. Removals **lag** additions (inventories grow by 35% in Kubernetes, by 13% in GitLab).
3. Lifecycles are **project dependent** (with a median of 734 days against 185 days).
4. Some toggles become **de facto permanent** (8 in Kubernetes and 25 in GitLab).

On top of that, we provide the **first benchmarking framework** for toggle management, with five metrics, empirical thresholds, and a live dashboard.

Replication package: doi.org/10.5281/zenodo.18773811